

Read Concern

The `readConcern` option allows you to control the consistency and isolation properties of the data read from replica sets and sharded clusters.

Through the effective use of write concerns and read concerns, you can adjust the level of consistency and availability guarantees as appropriate, such as waiting for stronger consistency guarantees, or loosening consistency requirements to provide higher availability.

Replica sets and sharded clusters support setting a global default read concern. Operations which do not specify an explicit read concern inherit the global default read concern settings.

See `setDefaultRWConcern` for more information.

Read Concern Levels

The following read concern levels are available:

Level	Description
"local"	The query returns data from the instance with no guarantee that the data has been written to a majority of the replica set members (i.e. may be rolled back). Default for reads against the primary and secondaries. Availability: Read concern "local" is available for use with or without causally consistent sessions and transactions. For more information, see the "local" reference page.
"available"	The query returns data from the instance with no guarantee that the data has been written to a majority of the replica set members (i.e. may be rolled back). Availability: Read concern "available" is unavailable for use with causally consistent sessions and transactions. For sharded clusters, "available" read concern provides the lowest latency reads possible among the various read concerns. However, this comes at the expense of consistency as "available" read concern can return orphaned documents when reading from a sharded collection. To avoid the risk of returning orphaned documents when reading from sharded collections, use a different read concern such as read concern "local". For more information, see the "available" reference page.
"majority"	The query returns the data that has been acknowledged by a majority of the replica set members. The documents returned by the read operation are durable, even in the event of failure. To fulfill read concern "majority", the replica set member returns data from its in-memory view of the data at the majority-commit point. As such, read concern "majority" is comparable in performance cost to other read concerns. Availability: Read concern "majority" is available for use with or without causally consistent sessions and transactions.

Level

Description

Requirements: To use read concern level of "majority", replica sets must use WiredTiger storage engine.

NOTE

For operations in multi-document transactions, read concern "majority" provides its guarantees only if the transaction commits with write concern "majority". Otherwise, the "majority" read concern provides no guarantees about the data read in transactions.

For more information, see the "majority" reference page.

"linearizable"

The query returns data that reflects all successful majority-acknowledged writes that completed prior to the start of the read operation. The query may wait for concurrently executing writes to propagate to a majority of replica set members before returning results.

If a majority of your replica set members crash and restart after the read operation, documents returned by the read operation are durable if `writeConcernMajorityJournalDefault` is set to the default state of `true`.

With `writeConcernMajorityJournalDefault` set to `false`, MongoDB does not wait for `w: "majority"` writes to be written to the on-disk journal before acknowledging the writes. As such, "majority" write operations could possibly roll back in the event of a transient loss (e.g. crash and restart) of a majority of nodes in a given replica set.

Availability:

- Read concern "linearizable" is **unavailable for use** with causally consistent sessions and transactions.
- You can specify linearizable read concern for read operations on the `primary` only.

You cannot use the `$out` or the `$merge` stage in conjunction with read concern "linearizable". That is, if you specify "linearizable" read concern for `db.collection.aggregate()`, you cannot include either stages in the pipeline.

Requirements: Linearizable read concern guarantees only apply if read operations specify a query filter that uniquely identifies a single document.

TIP

Always use `maxTimeMS` with linearizable read concern in case a majority of data bearing members are unavailable. `maxTimeMS` ensures that the operation does not block indefinitely and instead ensures that the operation returns an error if the read concern cannot be fulfilled.

For more information, see the "linearizable" reference page.

"snapshot"

A query with read concern "snapshot" returns majority-committed data as it appears across shards from a specific single point in time in the recent past. Read concern "snapshot" provides its guarantees only if the transaction commits with write concern "majority".

If a transaction is not part of a causally consistent session, upon transaction commit with write concern "majority", the transaction operations are guaranteed to have read from a snapshot of majority-committed data.

If a transaction is part of a causally consistent session, upon transaction commit with write concern "majority", the transaction operations are guaranteed to have read from a snapshot of majority-committed data that provides causal consistency with the operation immediately preceding the transaction start.

Availability:

Read concern "snapshot" is available for

Level	Description
	- All read operations inside multi-document transactions with the read concern set at the transaction level. - The following methods outside of multi-document transactions: <code>find</code> <code>aggregate</code> <code>distinct</code> (on unsharded collections) All other read operations prohibit <code>"snapshot"</code>.

Regardless of the read concern level, the most recent data on a node may not reflect the most recent version of the data in the system.

For more information on each read concern level, see:

- Read Concern `"local"`
- Read Concern `"available"`
- Read Concern `"majority"`
- Read Concern `"linearizable"`
- Read Concern `"snapshot"`
- Default MongoDB Read Concerns/Write Concerns

readConcern Support

Read Concern Option

For operations not in multi-document transactions, you can specify a `readConcern` level as an option to commands and methods that support read concern:

```
readConcern: { level: <level> }
```



To specify the read concern level for `mongosh` method `db.collection.find()`, use the `cursor.readConcern()` method:

```
db.collection.find().readConcern(<level>)
```



Transactions and Available Read Concerns

For multi-document transactions, you set the read concern at the transaction level, not at the individual operation level. The operations in the transaction will use the transaction-level read concern. Any read concern set at the collection and database level is ignored inside the transaction. If the transaction-level read concern is explicitly specified, the client level read concern is also ignored inside the transaction.

IMPORTANT

Do not explicitly set the read concern for the individual operations. To set the read concern for transactions, see Read Concern/Write Concern/Read Preference.

You can set the read concern at the transaction start:

- For multi-document transaction, the following read concern levels are available:
 - "local"
 - "majority"
 - "snapshot"
- Write commands that are part of a multi-document transactions can support the transaction-level read concern.
- You can create collections and indexes inside a transaction. If explicitly creating a collection or an index, the transaction must use read concern "local". If you implicitly create a collection, you can use any of the read concerns available for transactions.

If unspecified at the transaction start, transactions use the session-level read concern or, if that is unset, the client-level read concern.

For more information, see Transaction Read Concern.

Causally Consistent Sessions and Available Read Concerns

For operations in a causally consistent session, "local", "majority", and "snapshot" levels are available. However, to guarantee causal consistency, you must use "majority". For details, see Causal Consistency.

Operations That Support Read Concern

The following operations support read concern:

IMPORTANT

To set read concern for operations in a transaction, you set the read concern at the transaction level, not at the individual operation level. Do not explicitly set the read concern for the individual operations in a transaction. For more information, see Transactions and Read Concern.

Command/Method	"local"	"available"	"majority"	"snapshot" [3]	"linearizable"
count	✓	✓	✓		✓
distinct	✓	✓	✓	✓ [2]	✓
find	✓	✓	✓	✓	✓
db.collection.find() via cursor.readConcern()	✓	✓	✓	✓	✓

Command/Method	"local"	"available"	"majority"	"snapshot" [3]	"linearizable"
getMore	✓				✓
aggregate db.collection.aggregate()	✓	✓	✓	✓	✓ [6]
Session.startTransaction()	✓		✓	✓	

[1] You cannot use the \$out or the \$merge stage in conjunction with read concern "linearizable". That is, if you specify "linearizable" read concern for db.collection.aggregate(), you cannot include either stages in the pipeline.

[2] Read concern "snapshot" is available only for certain read operations and for multi-document transactions. In a transaction, you cannot use the distinct command or its helpers on a sharded collection.

The following write operations can also accept a read concern if part of a multi-document transaction.

IMPORTANT

To set read concern for operations in a transaction, you set the read concern at the transaction level, not at the individual operation level.

Command	"local"	"available"	"majority"	"snapshot" [3]	"linearizable"
delete db.collection.deleteOne() db.collection.deleteMany() db.collection.remove()	✓			✓	
findAndModify db.collection.findAndModify() db.collection.findOneAndDelete() db.collection.findOneAndReplace() db.collection.findOneAndUpdate()	✓			✓	
insert db.collection.insertOne() db.collection.insertMany()	✓			✓	
update db.collection.updateOne() db.collection.updateMany() db.collection.replaceOne()	✓			✓	
create db.createCollection() (Requires fCV 4.4 or greater)	✓				

```
createIndexes ✓  
db.collection.createIndex()  
db.collection.createIndexes()  
(Requires fCV 4.4 or greater)
```

[3] (1, 2) Read concern "snapshot" is available only for certain read operations and multi-document transactions. For transactions, you set the read concern at the transaction level. The transaction operations that support "snapshot" correspond to the CRUD operations available in transactions. For more information, see Transactions and Read Concern.

Read Concern not Supported on local Database

The local database does not support read concerns. MongoDB silently ignores any configured read concern for an operation on a collection in the local database.

Considerations

Read Your Own Writes

You can use causally consistent sessions to read your own writes, if the writes request acknowledgment.

Real Time Order

Combined with "majority" write concern, "linearizable" read concern enables multiple threads to perform reads and writes on a single document as if a single thread performed these operations in real time; that is, the corresponding schedule for these reads and writes is considered linearizable.

Performance Comparisons

Unlike "majority", "linearizable" read concern confirms with secondary members that the read operation is reading from a primary that is capable of confirming writes with { w: "majority" } write concern. [4] As such, reads with linearizable read concern may be significantly slower than reads with "majority" or "local" read concerns.

Always use `maxTimeMS` with linearizable read concern in case a majority of data bearing members are unavailable. `maxTimeMS` ensures that the operation does not block indefinitely and instead ensures that the operation returns an error if the read concern cannot be fulfilled.

For example:

```
db.restaurants.find( { _id: 5 }  
) .readConcern("linearizable").maxTimeMS(10000)  
db.runCommand( {  
  find: "restaurants",  
  filter: { _id: 5 },
```



```
readConcern: { level: "linearizable" },
maxTimeMS: 10000
} )
```

- [4] In some circumstances, two nodes in a replica set may *transiently* believe that they are the primary, but at most, one of them will be able to complete writes with `{ w: "majority" }` write concern. The node that can complete `{ w: "majority" }` writes is the current primary, and the other node is a former primary that has not yet recognized its demotion, typically due to a network partition. When this occurs, clients that connect to the former primary may observe stale data despite having requested read preference `primary`, and new writes to the former primary will eventually roll back.

Read Operations and `afterClusterTime`

MongoDB includes support for causally consistent sessions. For read operations associated with causally consistent session, MongoDB supports the `afterClusterTime` read concern option to be set automatically by the drivers for operations associated with causally consistent sessions.

IMPORTANT

Do not manually set `afterClusterTime` for a read operation. MongoDB drivers set this value automatically for operations associated with causally consistent sessions. However, you can advance the operation time and the cluster time for the session, such as to be consistent with the operations of another client session. For an example, see Examples.

NOTE

It is not possible to specify `atClusterTime` in conjunction with `afterClusterTime`. To use `atClusterTime` with read concern `"snapshot"` you have to disable causally consistent sessions.

To satisfy a read request with an `afterClusterTime` value of `T`, a `mongod` must perform the request after its oplog reaches time `T`. If its oplog has not reached time `T`, the `mongod` must wait to service the request.

Read operations with a specified `afterClusterTime` return data that meet both the read concern level requirement and the specified `afterClusterTime` requirement.

For read operations not associated with causally consistent sessions, `afterClusterTime` is unset.

Read Concern Provenance

MongoDB tracks read concern `provenance`, which indicates the source of a particular read concern. You may see `provenance` shown in the `getLastError` metrics, read concern error objects, and MongoDB logs.

The following table shows the possible read concern `provenance` values and their significance:

Provenance	Description
<code>clientSupplied</code>	The read concern was specified in the application.
<code>customDefault</code>	The read concern originated from a custom defined default value. See <code>setDefaultRWConcern</code> .
<code>implicitDefault</code>	The read concern originated from the server in absence of all other read concern specifications.